

Software Requirements Specification (SRS) - Team 11

Project: Snake Game (using graphics or console-based interface)

Version: 1.0

Authors: S Prithika, S N Niharika, S V Bhavana, S Ashwin

SRN: PES2UG24CS415, PES2UG24CS414, PES2UG24CS417, PES2UG24CS413

Date: 04-09-2026

Status: Final

Revision history

Version	Date	Author	Change summary	Approval
1.0	04-09-2026	S Prithika, S N Niharika, S V Bhavana, S Ashwin	SRS with diagrams embedded	

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			

Table of Contents

1. Introduction
2. Overall description
3. External interfaces
4. System features (detailed)
5. Non-functional requirements (detailed)
6. Quality attributes & Acceptance tests
7. UML Use-Case Diagram
8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the functional and non-functional requirements for the Snake Game. The purpose of this document is to provide a clear and complete description of the expected behavior, features, interfaces, constraints, and quality requirements of the system.

The SRS serves as a reference for the project team during design, development, testing, and integration. It also provides a basis for verifying that the completed Snake Game satisfies the requirements defined for the project.

The system is designed as a standalone, single-player game in which the player controls a snake, collects food to increase its length and score, and avoids collisions with the game boundaries and its own body.

1.2 Scope

The Snake Game is a standalone, single-player application that allows a player to control a snake on a fixed-size game board. The player uses keyboard controls to change the direction of the snake while the game continuously updates the snake's position.

The main functions within the scope of the system include:

- Displaying a main menu when the application starts.
- Allowing the player to select a difficulty level.
- Initializing a new game with a predefined starting state.
- Controlling the snake using keyboard input.
- Continuously moving and redrawing the snake during gameplay.
- Randomly placing food on unoccupied grid cells.
- Increasing the snake's length when food is consumed.
- Updating and displaying the player's score.
- Detecting collisions with the game boundary and the snake's own body.
- Displaying a Game Over screen with the final score.
- Allowing the player to restart or return to the menu.
- Allowing the player to pause and resume an active game.
- Adjusting the snake's movement speed according to the selected difficulty.
- Providing an Administrator role that can authenticate and access protected game data and stored score/configuration information.
- Restricting administrative and configuration operations to an authenticated Administrator.

The project does not include network communication, multiplayer functionality, or external service integration. The system is intended to operate as an offline application. This is consistent with the defined project scope for a standalone, single-player Snake Game.

1.3 Audience

This document is intended for:

- **Project Developers:** To understand the required functionality and system behavior during implementation.
- **Testers/QA Members:** To create and execute test cases based on the defined acceptance criteria.
- **Project Team Members:** To coordinate the development of different system features and ensure that the modules work together correctly.
- **Course Coordinator/Instructor:** To evaluate whether the project satisfies the specified software requirements.
- **Assessment Evaluators:** To verify the functional and non-functional requirements of the completed system.

1.4 Definitions

The following terms are used throughout this document:

Term	Definition
Snake	The player-controlled object that moves around the game board and grows when it consumes food.
Food	An item placed on an unoccupied grid cell that the snake can consume to increase its length and score.
Game Board	The fixed-size grid area in which the snake moves and food is placed.
Game Tick	A single update cycle during which the game state and snake position are updated.

Score	The points earned by the player by consuming food.
Game Over	The state reached when the snake collides with a wall or itself, according to the game rules.
Difficulty	A setting that changes the gameplay speed or challenge level.
HUD	Heads-Up Display showing information such as the player's score and game status.
Functional Requirement (FR)	A requirement describing what the system must do.
Non-Functional Requirement (NFR)	A requirement describing qualities or constraints of the system, such as performance and usability.
UML	Unified Modeling Language used to represent system design and behavior.
RTM	Requirements Traceability Matrix used to map requirements to their corresponding test cases.

2. Overall description

The Snake Game is a standalone, single-player application. The system does not depend on external network services and operates locally on a computer.

The system consists of the following main components/modules:

- MenuModule – part of User Interface; handles the Main Menu, difficulty selection, and navigation.

- **RenderModule / UI** – part of User Interface and Rendering/Output; handles the HUD, Game Over screen, and on-screen redraws.
- **InputModule** – part of Game Control Logic; handles keyboard input and directional control.
- **GameCore** – part of Game State Management; handles snake position, food, score, collisions, and game state.
- **PersistenceModule** – part of Game State Management; handles stored score and configuration data.
- **AdminModule** – part of Game Control Logic; handles Administrator authentication and administrative operations.
- **ValidationModule** – part of Game Control Logic; handles input and data validation.

2.2 Major Product Functions

The major functions of the Snake Game are:

1. **Menu and Initialization** – provides the main menu and starts a new game.
2. **Movement and Control** – allows the player to control the snake using keyboard inputs.
3. **Food and Growth** – generates food and increases the snake's length when food is consumed.
4. **Collision and Game Over** – detects collisions with the game boundaries or the snake itself and ends the game when required.
5. **Scoring** – increases and displays the player's score when food is consumed.
6. **Pause and Resume** – allows the player to pause and continue the game without losing the current game state.
7. **Difficulty Selection** – provides different difficulty levels that affect the speed or challenge of the game.
8. **Administration and Access Control** – provides an authenticated Administrator role that can access protected game data and stored score/configuration information, with administrative and configuration operations restricted to authorized access only.

2.3 User Roles and Characteristics

The primary user of the system is the **Player**.

Player

The player controls the snake using the keyboard, selects game options, consumes food, attempts to achieve a high score, and can pause, resume, restart, or exit the game as supported by the system.

No specialized technical knowledge is required to play the game.

Administrator

The Administrator is a restricted role responsible for accessing protected game data and performing administrative or configuration operations, including modifying stored score and configuration information. The Administrator must be authenticated before any protected data or administrative/configuration operation can be accessed.

Administrator access is separate from normal gameplay and is not required, or available, to the Player.

2.4 Operating Environment

The Snake Game is intended to run on a desktop or laptop computer with:

- A display capable of supporting a minimum resolution of 800 × 600 pixels.
- A keyboard for player input.
- A supported operating system and runtime environment.
- A suitable graphics/rendering or game-loop library.
- Keyboard input APIs supported by the selected development environment.

The primary keyboard controls are expected to include:

- **Arrow keys / WASD** – control snake movement.
- **Enter / Space** – select or confirm menu options where applicable.
- **Esc** – pause, return, or exit where applicable.

The game does not require an internet connection and is designed to operate as a standalone application.

2.5 Constraints

The Snake Game is subject to the following constraints:

- The game is designed for single-player use.
- The game operates offline without requiring network communication.
- Player interaction is primarily through keyboard input.
- Snake movement must follow the defined game-loop timing.
- The system should provide responsive input and stable game updates.
- The game should maintain consistent rendering and gameplay during normal operation.
- The implementation is dependent on the selected programming language, graphics/game framework, and operating-system input APIs.

- The user interface and gameplay behavior should remain consistent with the functional and non-functional requirements specified in this SRS.

3. External interface requirements

3.1 User Interfaces

The game presents three player-facing screens: a Main Menu (Start Game, Difficulty Select, Exit), an in-game HUD overlaying the board (current score, and on higher difficulties the active speed level), and a Game Over screen (final score, Restart / Return to Menu options). All screens use a single consistent color/typography scheme and are operable entirely by keyboard, with the currently focused option always visually highlighted for accessibility.

3.2 Hardware Interfaces

- Keyboard: Arrow keys and WASD for directional control; Enter/Space to confirm menu selections; Esc to pause
- Display: standard monitor/screen output via the rendering surface, minimum resolution assumed 800×600.

3.3 Software Interfaces

- Rendering/game-loop library or framework used to draw the board, snake, food, and menu screens each frame.
- Operating system windowing/input APIs used to receive raw keyboard events and forward them to the game's input handler.

3.4 Communications

Not applicable. Snake is implemented as a standalone, single-player, offline application with no network, multiplayer, or external service communication in this project's scope.

3.5 Administrator Interface

The system provides a separate, authenticated entry point for the Administrator, distinct from the Player-facing Main Menu, HUD, and Game Over screens described in Section 3.1. Access to protected game data, stored score/configuration information, and administrative/configuration operations is granted only after successful Administrator authentication. This interface is not exposed to, or usable by, the Player during normal gameplay.

4. System features (detailed)

Each requirement below includes acceptance criteria and a reference test case.

4.1 Game initialisation and Menu

On launch the player is shown a Main Menu where they choose a difficulty and start a new game, or exit. Selecting Start Game triggers game initialization: a fixed-size board is created, the snake is placed at a default position with a starting length of 3 segments moving in a default direction, the score is reset to zero, and the first food item is placed on the board (see Section 4.3 for food placement logic). This section produces a fully initialized, ready-to-play game state consumed by the gameplay mechanics in Sections 4.2–4.4.

Req ID	Requirement	Type	Priority	Source/ Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SNK-F-001	The system shall display a Main Menu on application launch, presenting Start Game, Difficulty Select, and Exit options.	Functional	High	player	AC-SNK-F-001: On launch, the Main Menu appears within 1s showing all three options; each is selectable via keyboard (Up/Down + Enter). Test: TC-Menu-01	A discoverable entry point is required before any gameplay can begin; this is the first thing every player encounters.
SNK-F-002	The system shall allow the player to select a difficulty level (Easy / Medium / Hard) from the Main Menu before starting a game. .	Functional	Medium	player	AC-SNK-F-002: Selecting a difficulty highlights it and stores the choice; the choice is passed to game initialization (affecting starting speed). Test: TC-Menu-02	Difficulty selection is a common player-facing feature and its output (chosen speed tier) is consumed directly by initialization, so it belongs with the menu flow.
SNK-F-003	On selecting Start Game, the system shall initialize the game board, place	Functional	High	player	AC-SNK-F-003: A new game begins with a board of the configured size, a snake of	A consistent, deterministic starting state is required so every game

Req ID	Requirement	Type	Priority	Source/ Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
	the snake at a default starting position/length, and reset the score to zero.				length 3 centered on the board, and score = 0 displayed on the HUD. Test: TC-Init-01	and every acceptance test begins from the same conditions

4.2 Movement and Control

Description: Move the snake each tick in its current direction and respond to player input. This subsection is split between the two members responsible for gameplay flow.

Req ID	Requirement (shall...)	Type	Priority	Source/ Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SNK-F-004	Accept arrow-key input to change the snake's direction, not allowing a 180° reversal into its own neck	Functional	High	Player	Pressing the opposite direction is ignored; all other directions apply. Test: TC-Move-02	
SNK-F-005	Redraw the snake and grid on every tick to reflect the updated position	Functional	Medium	Player	Visual/console output matches internal grid state after each tick. Test: TC-Move-03	SFML draw call / console repaint
SNK-F-006	The system shall allow the player to return to the Main Menu or exit the application from the Main Menu screen using	Functional	Low	player	AC-SNK-F-006: Selecting Exit from the Main Menu closes the application cleanly; no mouse input is required at any menu step. Test: TC-Menu-03	Keeps the menu system self-contained and fully operable without additional input hardware, consistent with the rest of the control scheme.

	keyboard input only.					
--	----------------------	--	--	--	--	--

4.3 Food and Growth

Description: Spawn food randomly and grow the snake when it is eaten.

Req ID	Requirement (shall...)	Type	Priority	Source/S takeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SNK-F-007	randomly place one food item on a grid cell not currently occupied by the snake	Functional	High	Player	food never spawns on a snake segment. Test: TC-Food-01	Needs occupied-cell check
SNK-F-008	detect when the snake's head enters the food's cell, grow the snake by one segment, and immediately spawn a new food item.	Functional	High	Player	snake length increases by 1 on contact and a new food cell appears the same tick. Test: TC-Food-02, TC-Food-03	

4.4 Collision and Game over

Description: End the game when the snake collides with a wall or itself, and show the result.

Req ID	Requirement (shall...)	Type	Priority	Source/S takeholder	Acceptance criteria / Test case ref	Comments / Dependencies
SNK-F-009	detect collision between the snake's head and either the grid boundary or any other segment of its own body,	Functional	High	Player	Game transitions to "Game Over" the tick either collision type occurs. Test: TC-Collision-01, TC-Collision-02	

	and end the game.					
SNK-F-010	display a "Game Over" screen showing the final score with options to Restart or Exit.	Functional	High	Player	Final score matches last recorded score; both options are selectable. Test: TC-Collision-03	

4.5 Scoring

Description: The system shall maintain and display player's score based on the gameplay progress. The system shall update score when the player successfully collects food and shall display final score when game ends. System shall display a single local player high score.

Req ID	Requirement	Type	Priority	Source/Shareholder	Acceptance criteria / Test case	Comments / Dependencies
SNK-F-011	The system shall increase the player's score whenever the snake successfully consumes food.	Functional	High	Player	AC-SG-F-011: When the snake successfully consumes one food item, the displayed score shall increase according to the defined scoring rule. Test: TC-Score-01	Depends on food generation and collision detection with the food item.
SNK-F-012	The system shall display the player's final score when the game ends.	Functional	High	Player	AC-SG-F-012: When the game ends because of a game-over condition, the system shall display the final score along with the game-over information. Test: TC-Score-01	Depends on the game-over mechanism and score-tracking functionality.

4.6 Pause / Difficulty

Description: The system shall allow the player to pause and resume an active game without losing the current game state. The system shall also allow the player to select a difficulty level before starting the game, with the selected difficulty determining the snake's movement speed.

Req ID	Requirement	Type	Priority	Source/Shareholder	Acceptance criteria / Test case	Comments / Dependencies
SNK-F-013	The system shall allow the player to pause an active game and resume the game without losing the current game state.	Functional	High	Player	AC-SG-F-013: When the player selects the pause option during an active game, snake movement shall stop. When the player selects resume, the game shall continue from the same game state and snake position at which it was paused. Test: TC-Pause-01	Depends on active game-state management and input handling.
SNK-F-014	The system shall allow the player to select a difficulty level before starting a game, and the selected difficulty shall determine the snake's movement speed.	Functional	Medium	Player	AC-SG-F-014: The player shall be able to select one of the available difficulty levels before starting the game. The snake shall move at the speed associated with the selected difficulty level during gameplay. Test: TC-Difficulty-01	Depends on game initialization and movement-speed control.

4.7 Administrator Functions

Description: Provide a restricted, authenticated Administrator role that can access protected game data and perform administrative or configuration operations, including modifying stored score/configuration information.

Req ID	Requirement	Type	Priority	Source/S takehold er	Acceptance criteria / Test case	Comments / Dependencies
SNK-F-015	The system shall authenticate the Administrator before granting access to any protected game data or administrative/co nfiguration operation.	Func ti onal	High	Administ rator	AC-SNK-F-015: Access to protected data or administrative operations is denied until the Administrator successfully authenticates; failed authentication does not grant access. Test: TC-Admin-01	Required by SNK-F-016, SNK-F-017, and SNK-F-018 via <<include>>.
SNK-F-016	The system shall allow an authenticated Administrator to access protected game data.	Func ti onal	Medium	Administ rator	AC-SNK-F-016: Protected game data is viewable only after successful Administrator authentication; access is denied otherwise. Test: TC-Admin-02	Depends on SNK-F-015 (Authenticate Administrator).
SNK-F-017	The system shall allow an authenticated Administrator to modify stored score/configurati on data.	Func ti onal	Medium	Administ rator	AC-SNK-F-017: Stored score/configuration values can be changed only by an authenticated Administrator; the modified values are reflected correctly afterward. Test: TC-Admin-03	Depends on SNK-F-015 (Authenticate Administrator).
SNK-F-018	The system shall allow an authenticated Administrator to perform administrative/co nfiguration operations on the system.	Func ti onal	Medium	Administ rator	AC-SNK-F-018: Administrative/config uration operations complete successfully for an authenticated Administrator and are rejected for an unauthenticated user. Test: TC-Admin-04	Depends on SNK-F-015 (Authenticate Administrator).

5. Non-functional requirements (detailed)

NFRs below are measurable and tied to test plans.

Req ID	Requirement	Category	Priority	Acceptance criteria / Measurement
SNK-NF-001	The system shall register and apply a directional keypress within one game tick (≤ 100 ms at the slowest difficulty) of the key being pressed. .	Performance	High	AC-SNK-NF-001: Input-to-response latency measured at ≤ 100 ms in 95% of sampled keypresses during a test session. Test: TC-Perf-Ctrl-01
SNK-NF-002	The Main Menu and difficulty selection shall be fully navigable using only the keyboard, with the currently selected option always visibly highlighted	Usability	Medium	AC-SNK-NF-002: A user with no prior instructions can select a difficulty and start a game using only arrow keys and Enter in a usability walkthrough. Test: TC-UX-Menu-01
SNK-NF-003	The game loop shall maintain a stable update/redraw rate (≥ 30 FPS in SFML mode, or a consistent tick interval in console mode) under normal play.	Performance	High	Frame-time logging shows ≥ 30 FPS / tick jitter $< 10\%$. Test: TC-Perf-01
SNK-NF-004	The application shall run for at least 30 minutes of continuous play without crashing or memory leaks.	Reliability	High	Soak test of 30+ minutes completes with no crash; memory usage stable. Test: TC-Rel-01
SNK-NF-005	The system shall respond to player input within 200 milliseconds during normal gameplay.	Performance	High	AC-SG-NF-005: During normal gameplay, the delay between a valid player input and the corresponding change

				in snake movement direction shall not exceed 200 milliseconds. Test: TC-Perf-01
SNK-NF-006	The system shall provide clearly distinguishable controls and readable game-status information for starting, pausing, resuming, and restarting the game.	Usability	Medium	AC-SG-NF-006: During usability testing, players shall be able to identify the controls for starting, pausing, resuming, and restarting the game without developer assistance. Test: TC-UX-01

5.1. Security

5.1.1 Security Objectives

Security Objective 1

SO-001: The system shall protect player-related data including scores, from unauthorized modification or deletion.

Security Objective 2

SO-002: The system shall prevent unauthorized users or processes from modifying game configuration, stored scores, or other persistent game information.

5.1.2 Security Requirements

Req ID	Requirement (shall...)	Type	Priority	Acceptance criteria / Test case ref
SNK-SR-001	The system shall validate player-provided input, including game configuration values, before storing or processing the input.	Security	High	AC-SG-SR-001: Invalid, excessively long, or unsupported input shall be rejected without causing the game to crash or modify stored game data. Test: TC-Sec-01

SNK-SR-002	The system shall prevent unauthorized modification of stored player scores.	Security	High	AC-SG-SR-002: Attempts by an unauthorized user or process to modify stored scores shall be rejected, and the original records shall remain unchanged. Test: TC-Sec-02
SNK-SR-003	System shall restrict administrative/configuration operations to authorized users/processes.	Security	Medium	Unauthorized access denied. TC-Sec-03
SNK-SR-004	The system shall protect stored player information and game configuration data from unauthorized access or disclosure.	Security	Medium	AC-SG-SR-004: Unauthorized users shall not be able to access stored player information or game configuration data. Authorized administrators shall be able to access only the data and functions permitted to them. Test: TC-Sec-04

6. Quality attributes & Acceptance tests

6.1 Interfaces and Basic Gameplay acceptance

Test Case ID	Requirement ID	Test Objective	Test Procedure	Expected Result
TC-Menu-01	SNK-F-001	Verify that the Main Menu is displayed correctly and its options are selectable.	1. Launch the application. 2. Observe the Main Menu. 3. Use Up/Down keys to navigate. 4. Press Enter to select an option.	Main Menu appears within 1 second with Start Game, Difficulty Select, and Exit options. Options can be selected using the keyboard.
TC-Menu-02	SNK-F-002	Verify that the player can select a difficulty level before starting the game.	1. Launch the application. 2. Navigate to Difficulty Select. 3. Select Easy, Medium, or Hard.	Selected difficulty is visibly highlighted and stored. The selected difficulty is passed to game initialization and affects the starting

			4. Confirm the selection.	speed.
TC-Init-01	SNK-F-003	Verify that a new game starts with the correct initial state.	1. Launch the application. 2. Select a difficulty. 3. Select Start Game. 4. Observe the board, snake, and score.	Game starts with the configured board size, snake has 3 segments at the center/default position, and score = 0 is displayed on the HUD
TC-Ctrl-01	SNK-F-004	Verify that keyboard inputs correctly control the snake's intended direction.	1. Start a game. 2. Press each Arrow key and WASD direction key. 3. Observe the direction requested by the player before the next movement tick. 4. Press an unmapped key.	All 8 mapped directional keys correctly update the intended direction. Unmapped keys are ignored.
TC-Ctrl-02	SNK-F-004	Verify that direct opposite-direction inputs are rejected.	1. Start the snake moving in one direction, e.g. Up. 2. Immediately press the opposite direction, Down. 3. Observe the snake's direction on the next tick	Opposite-direction input is discarded. The snake continues moving Up and does not reverse or self-collide due to that input
TC-Menu-03	SNK-F-006	Verify that the application can be exited using keyboard input only	1. Launch the application. 2. Navigate to the Main Menu using the keyboard. 3. Select Exit using keyboard input.	Application closes cleanly without requiring mouse input.
TC-Perf-Ctrl-01	SNK-NF-001	Verify that directional input is processed within the required response time.	1. Start a game. 2. Record multiple directional keypresses and their response times. 3. Calculate the percentage of inputs processed within 100 ms.	Input-to-response latency is ≤ 100 ms for at least 95% of sampled keypresses.

TC-UX-Menu-01	SNK-NF-002	Verify that the Main Menu and difficulty selection are fully usable with keyboard-only controls	1. Start the application without using a mouse. 2. Navigate through the Main Menu using Arrow keys. 3. Select difficulty. 4. Start the game using Enter. 5. Observe the selected-option highlight throughout	User can navigate and start a game using only Arrow keys and Enter. The currently selected option is always visibly highlighted.
---------------	------------	---	--	--

6.2 Core gameplay acceptance

Test Case ID	Requirement ID	Test Objective	Test Procedure	Expected Result
TC-Move-02	SNK-F-004	Verify direction changes are accepted and a 180° reversal is rejected.	1. Start a game. 2. Press a valid perpendicular direction key. 3. Observe the snake's heading. 4. While moving, press the key for the exact opposite direction. 5. Observe the snake's heading again.	The snake's direction shall update on the valid perpendicular key press. The opposite-direction key press shall be ignored, and the snake shall continue in its current direction.
TC-Move-03	SNK-F-005	Verify the grid and snake are redrawn correctly on every tick.	1. Start a game. 2. Let the snake move for several ticks. 3. Compare the on-screen/console grid state to the internal game state after each tick.	The displayed snake position and grid shall match the internal game state after every tick, with no lag or stale frames.
TC-Food-01	SNK-F-007	Verify food spawns only on unoccupied grid cells.	1. Start a game. 2. Let the snake occupy several cells. 3. Trigger multiple food	No spawned food coordinate shall ever coincide with a cell currently

			spawn events (by repeatedly eating). 4. Record the coordinates of each spawned food item.	occupied by the snake.
TC-Food-02	SNK-F-008	Verify the snake grows by one segment when it eats food.	1. Start a game. 2. Note the snake's current length. 3. Move the snake onto the food cell. 4. Observe the snake's length after contact.	The snake's length shall increase by exactly one segment immediately after the head reaches the food cell.
TC-Food-03	SNK-F-008	Verify a new food item spawns immediately after the previous one is eaten.	1. Start a game. 2. Move the snake onto the food cell. 3. Observe the grid immediately after consumption.	A new food item shall appear on the grid within the same tick the previous food item was consumed.
TC-Collision-01	SNK-F-009	Verify the game ends when the snake's head hits a wall.	1. Start a game. 2. Steer the snake directly into a grid boundary. 3. Observe game state on impact.	The game shall transition to Game Over the tick the snake's head crosses the boundary.
TC-Collision-02	SNK-F-009	Verify the game ends when the snake's head hits its own body.	1. Start a game. 2. Grow the snake to at least 5 segments. 3. Steer the snake so its head overlaps a body segment. 4. Observe game state on impact.	The game shall transition to Game Over the tick the snake's head overlaps any of its own body segments.
TC-Collision-03	SNK-F-010	Verify the Game Over screen displays the correct final score with working Restart/Exit options.	1. Trigger a Game Over via wall or self-collision. 2. Observe the Game Over screen. 3. Select Restart. 4. Relaunch and trigger Game Over again, then select Exit.	The Game Over screen shall show the final score matching the last recorded in-game score. Restart shall start a new game; Exit shall close the application.

TC-Perf-01	SNK-NF-003	Verify the game loop maintains a stable frame/tick rate under normal play.	1. Start a game. 2. Play for at least 5 minutes under normal conditions. 3. Log frame time / tick interval throughout.	The measured rate shall stay ≥ 30 FPS (SFML) or hold a consistent tick interval (console) with jitter under 10%.
TC-Rel-01	SNK-NF-004	Verify the application runs for 30+ minutes without crashing or leaking memory.	1. Start a game. 2. Leave it running/playing continuously for at least 30 minutes. 3. Monitor memory usage and application stability throughout.	The application shall not crash, and memory usage shall remain stable (no continuous growth) over the full duration.

6.3 Advanced features, Security, UML acceptance

Test Case ID	Requirement ID	Test Objective	Test Procedure	Expected Result
TC-Score-01	SNK-F-011	Verify that the player's score increases when food is consumed.	1. Start a game. 2. Allow the snake to move toward a food item. 3. Make the snake consume the food. 4. Observe the displayed score.	The displayed score shall increase according to the defined scoring rule after the snake successfully consumes the food item.
TC-FinalScore-01	SNK-F-012	Verify that the final score is displayed when the game ends.	1. Start a game. 2. Accumulate a score by consuming food. 3. Trigger a game-over condition. 4. Observe the game-over screen/message.	The system shall display the player's final score together with the game-over information.
TC-Pause-01	SNK-F-013	Verify that the game can be paused and resumed without losing the current	1. Start a game. 2. Allow the snake to move. 3. Pause the game. 4. Observe the snake. 5. Resume the game. 6. Compare the game state	Snake movement shall stop while paused. After resuming, the game shall continue from the same game state and snake position at which it was

		game state.	before and after pausing.	paused.
TC-Difficulty-01	SNK-F-014	Verify that the selected difficulty determines the snake's movement speed.	1. Open the game. 2. Select an available difficulty level. 3. Start the game. 4. Observe the snake's movement speed. 5. Repeat the test using the other available difficulty levels.	The player shall be able to select a difficulty before starting the game, and the snake shall move at the speed associated with the selected difficulty level.
TC-InputResp-01	SNK-NF-005	Verify the response time of the system to player input during normal gameplay.	1. Start a normal game. 2. Provide a valid directional input. 3. Measure the time between the input and the corresponding change in snake movement direction. 4. Repeat for multiple valid inputs.	The system shall respond to valid player input within 200 milliseconds during normal gameplay.
TC-UX-01	SNK-NF-006	Verify that game controls and status information are clearly distinguishable and understandable.	1. Start the game. 2. Ask a representative user to identify the controls for starting, pausing, resuming, and restarting the game. 3. Observe whether the user can use the controls without developer assistance.	The controls shall be clearly distinguishable, and the user shall be able to identify and use the start, pause, resume, and restart controls without developer assistance.
TC-Sec-01	SNK-SR-001	Verify that player-provided input and game configuration values are properly validated.	1. Enter valid input. 2. Enter invalid input. 3. Enter excessively long input. 4. Enter unsupported input or configuration values. 5. Observe system behavior and stored data.	Valid input shall be accepted. Invalid, excessively long, or unsupported input shall be rejected without causing a system crash or unauthorized modification of stored data.
TC-Sec-02	SNK-SR-002	Verify that unauthorized modification of stored player scores is prevented.	1. Create a valid player score. 2. Attempt to modify the stored score without authorization. 3. Check the stored score after the attempt.	The unauthorized modification attempt shall be rejected, and the original stored score shall remain unchanged.

TC-Sec-03	SNK-SR-001	Verify that score data is validated before being stored or updated.	1. Submit valid score data. 2. Submit a negative score. 3. Submit malformed or otherwise invalid score data. 4. Check the stored score data.	Valid score data shall be accepted. Negative, malformed, or otherwise invalid score data shall be rejected and shall not be stored.
TC-Sec-04	SNK-SR-003	Verify that administrative or configuration operations are restricted to authorized users or processes.	1. Attempt to access an administrative or configuration operation without authorization. 2. Repeat the operation using an authorized administrator.	Unauthorized access shall be denied. An authorized administrator shall be permitted to perform only the administrative or configuration operations allowed by the system.
TC-Sec-05	SNK-SR-004	Verify stored player information and game configuration data cannot be accessed or disclosed to unauthorized users.	1. Attempt to access stored player/configuration data without authorization. 2. Observe system behavior. 3. Authenticate as an authorized Administrator. 4. Attempt to access only the data/functions permitted to that role.	Unauthorized access attempts are denied and no data is disclosed. An authenticated Administrator can access only the data and functions they are permitted to use.
TC-Admin-01	SNK-F-015	Verify that the Administrator must be authenticated before accessing protected game data or administrative /configuration operations.	Test Procedure: 1. Attempt to access protected game data or an administrative/configuration operation without authenticating. 2. Observe system behavior. 3. Authenticate as the Administrator. 4. Repeat the access attempt.	Access is denied when the Administrator is not authenticated. After successful authentication, access to protected data and administrative/configuration operations is granted.
TC-Admin-02	SNK-F-016	Verify that an authenticated Administrator	Test Procedure: 1. Authenticate as the Administrator. 2.	Expected Result: Protected game data is displayed correctly to the

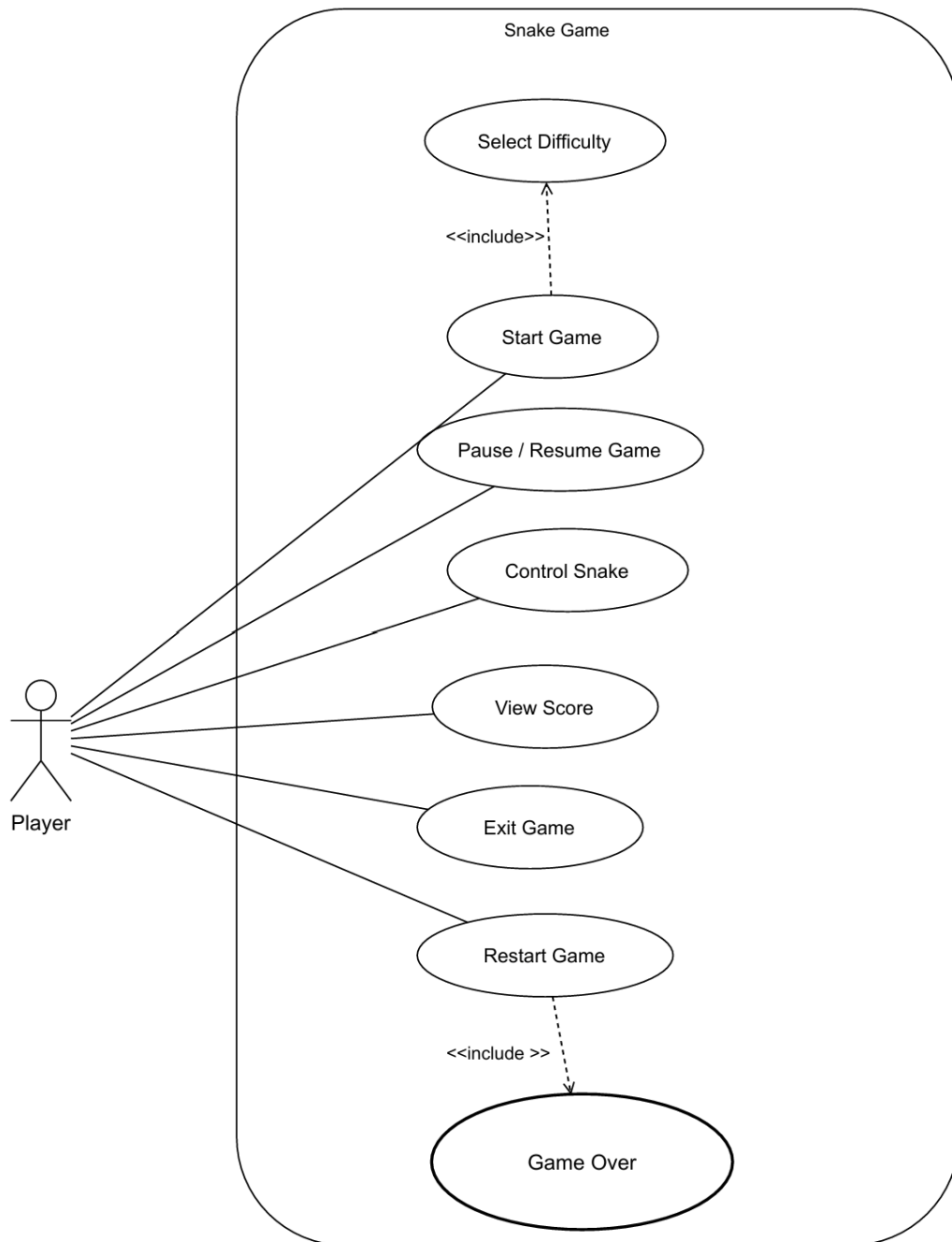
		can access protected game data.	Navigate to protected game data. 3. Observe the displayed data.	authenticated Administrator.
TC-Admin-03	SNK-F-017	Verify that an authenticated Administrator can modify stored score/configuration data.	Test Procedure: 1. Authenticate as the Administrator. 2. Modify a stored score or configuration value. 3. Save the change. 4. Check the stored data after the change.	The stored score/configuration data is updated correctly to reflect the Administrator's change.
TC-Admin-04	SNK-F-018	Verify that administrative /configuration operations can be performed by an authenticated Administrator and are rejected otherwise.	Test Procedure: 1. Attempt an administrative/configuration operation without authenticating. 2. Observe the result. 3. Authenticate as the Administrator. 4. Repeat the same operation.	The operation is rejected when unauthenticated. After successful authentication, the operation completes successfully.

7. System models and diagrams

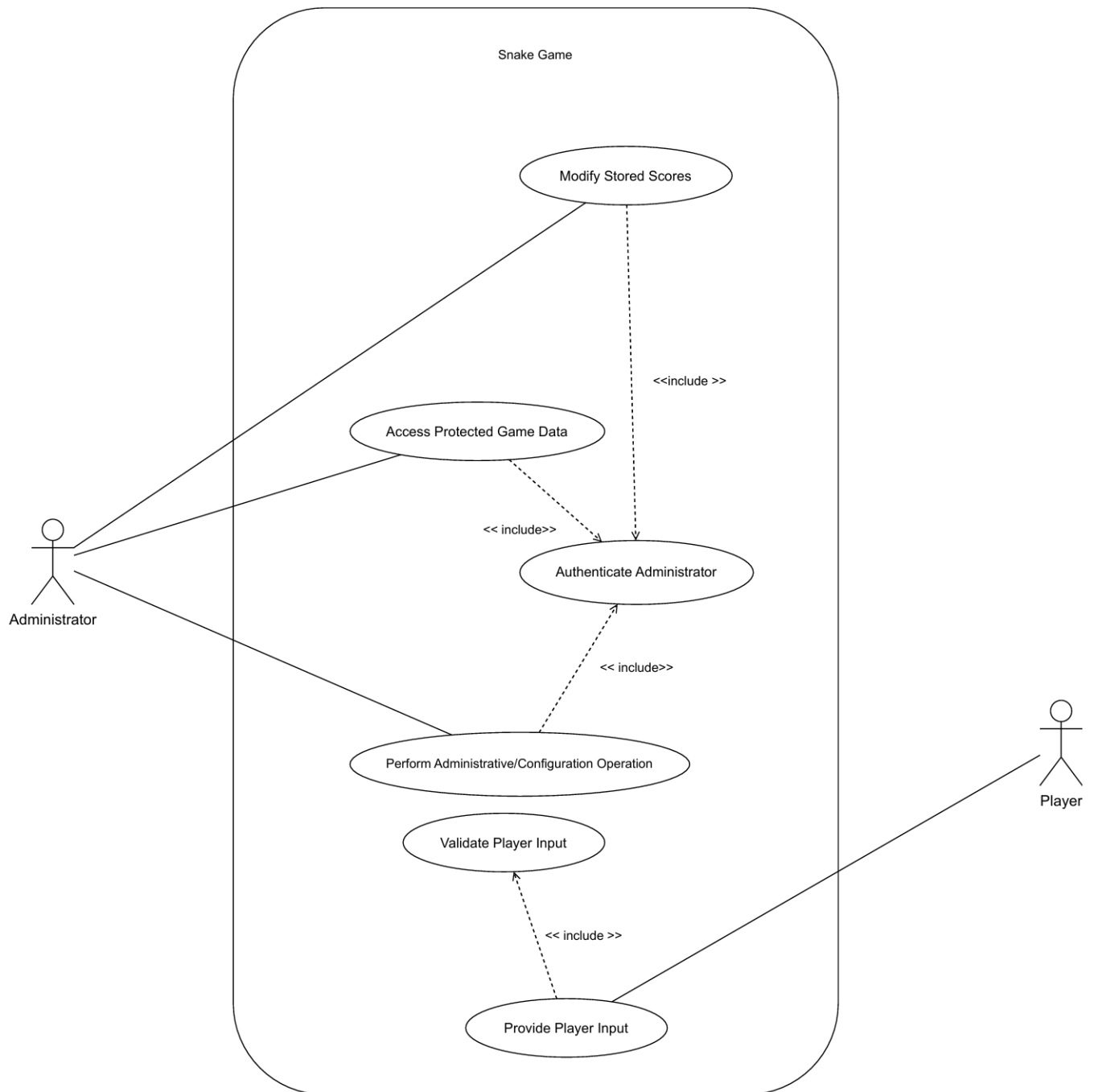
7.1 UML Use-Case diagram

At least 2 UML use case diagrams to be created.

UML1:



UML2:



8. Requirements Traceability Matrix (RTM)

Req ID	Requirements	Section	Module	Test Case(s)	Status(N/P/A)
SNK-F-001	Main menu	4.1	MenuModule	TC-Menu-01	N
SNK-F-002	Difficulty selection	4.1	MenuModule	TC-Menu-02	N
SNK-F-003	Grid/snake init	4.1	GameCore	TC-Init-01	N
SNK-F-004	Direction input	4.2	InputModule	TC-Move-02	N
SNK-F-005	Redraw each tick	4.2	RenderModule	TC-Move-03	N
SNK-F-006	Return to menu / exit via keyboard	4.2	MenuModule	TC-Menu-03	N
SNK-F-007	Food spawn	4.3	GameCore	TC-Food-01	N
SNK-F-008	Growth + respawn	4.3	GameCore	TC-Food-02, TC-Food-03	N
SNK-F-009	Wall/self collision	4.4	GameCore	TC-Collision-01, TC-Collision-02	N
SNK-F-010	Game Over screen	4.4	RenderModule	TC-Collision-03	N
SNK-F-011	Score incr. + display	4.5	GameCore	TC-Score-01	N
SNK-F-012	Final score displayed at game end	4.5	RenderModule	TC-FinalScore-01	N
SNK-F-013	Pause/resume	4.6	GameCore	TC-Pause-01	N
SNK-F-014	Speed scaling + difficulty select	4.6	GameCore / MenuModule	TC-Difficulty-01	N
SNK-F-015	Authenticate Administrator	4.7	AdminModule	TC-Admin-01	N
SNK-F-016	Authenticated admin accesses protected data	4.7	AdminModule	TC-Admin-02	N
SNK-F-017	Authenticated admin modifies stored score/config	4.7	AdminModule	TC-Admin-03	N
SNK-F-018	Authenticated admin performs config operations	4.7	AdminModule	TC-Admin-04	N
SNK-NF-001	Input registered within 100ms	5	InputModule	TC-Perf-01	N
SNK-NF-002	Menu/difficulty fully keyboard-navigable	5	MenuModule	TC-UX-Menu-01	N
SNK-NF-003	Stable FPS/tick rate	5	GameCore	TC-Perf-01	N

SNK-NF-004	30-min stability	5	GameCore	TC-Rel-01	N
SNK-NF-005	Input-to-movement response	5	InputModule	TC-InputResp-01	N
SNK-NF-006	Distinguishable controls/status info	5	UI	TC-UX-01	N
SNK-SR-001	Validate player input/config values	5.1.2	ValidationModule	TC-Sec-01	N
SNK-SR-002	Prevent unauthorized score modification	5.1.2	PersistenceModule	TC-Sec-02	N
SNK-SR-003	Restrict admin/config ops to authorized users	5.1.2	AdminModule	TC-Sec-04	N
SNK-SR-004	Protect stored data from unauthorized access/disclosure	5.1.2	PersistenceModule	TC-Sec-05	N